Study Notes: Building a Menu

A menu program shows a list of options, lets the user pick one, does that job, and comes back to the menu until the user chooses to quit. It's a `while` loop with a `switch` inside it. From now on, most of your exercises are built as a menu, with each problem as one option.

The whole pattern

```
const string MENU = """
    ****************************
    *  My Program              *
    *                          *
    *  A - Problem 1           *
    *  B - Problem 2           *
    *  Q - Quit                *
    ****************************
    """;

bool hasQuit = false;
string result = "";

while (!hasQuit)
{
    // 1. Draw the screen
    Console.Clear();
    Console.WriteLine(MENU);
    Console.WriteLine(result);
    Console.WriteLine();

    // 2. Read the selection
    Console.Write("Enter your selection: ");
    string selection = Console.ReadLine();
    selection = selection.ToUpper();

    // 3. Do the work
    switch (selection)
    {
        case "A":
            // ask for this problem's input, work it out
            result = "...the answer...";
            break;
        case "B":
            result = "...the answer...";
            break;
        case "Q":
            hasQuit = true;
            break;
        default:
            result = $"{selection} isn't an option. Please choose from the menu.";
            break;
    }
}

Console.WriteLine("Goodbye!");
```

The pieces

Piece	What it does
`const string MENU = """ ... """;`	The menu text. It never changes, so it's a constant (capitals). The triple quotes make a **raw string**: it prints exactly as typed, line breaks and all.
`bool hasQuit = false;`	Remembers whether the user has quit. Declared **before** the loop, because it must survive from one pass to the next.
`string result = "";`	Holds the answer to show under the menu. Also declared **before** the loop, for the same reason.
`while (!hasQuit)`	"While we haven't quit." Q is the **sentinel**: the special value that ends the loop.
`string selection`	Declared **inside** the loop. Every pass gets a brand-new one.
`ToUpper()`	Turns `b` into `B`, so lower-case letters work too.
`default`	Catches anything that isn't an option.

The menu rules

1. **The loop owns the screen.** At the top of every pass it clears the screen, draws the menu and shows `result`.
2. **The loop owns the selection.** "Enter your selection" is read in one place, at the top of the loop, and nowhere else.
3. **Each case owns one problem.** A case can ask for its own input (like "Bill amount:"), but it **never prints its answer** and **never asks for a menu selection**.
4. **A case sets `result` with `=`, once.** Never `+=` onto `result`: it still holds the last answer. If a case needs to build something up, it uses its own variable first, then puts the finished answer in `result`.

Why can't a case print its answer? Because the next pass starts with `Console.Clear()`. Anything a case prints is wiped before anyone can read it.

Building something up inside a case

Use the case's own variable, not `result`:

```
string countdown = "";

while (count > 0)
{
    countdown += $"{count}... ";
    count--;
}

result = countdown + "Liftoff!";
```

Adding a new option

Exactly two changes:

1. Add a line to the `MENU` text, like `* C - Problem 3 *`.
2. Add a `case "C":` above `case "Q":`.

Nothing else changes. That's what makes a menu **scale**, from three options to thirty.

One name, once per switch

A variable can only be used inside the code block (the pair of braces) where it's declared, and inside any blocks nested in it. That's why an accumulator like `sum` has to be declared **before** a loop: declared inside the loop's braces, the line after the loop can't see it.

A `switch` has one pair of braces around **all** of its cases, so every case is in the same block. That means a variable name can only be declared once in the whole switch. If case A and case D both declare `amount`, it won't build:

```
A local variable or function named 'amount' is already defined in this scope
```

Two ways to fix it:

- Give the second one a different name, like `kilometres`.
- If several cases need the same kind of value, declare it **once**, before the switch (`int number;`), and give it a value inside each case.

Common mistakes

What you see	Why	Fix
The answer flashes, then the menu comes back without it	`result` is printed after the switch, or a case printed its answer	Print `result` once, right after the menu
Q doesn't quit	Case Q only has `break`, which leaves the **switch**, not the loop	Set `hasQuit = true;` in case Q
A lower-case letter says it isn't an option	The cases are upper case, and text is case-sensitive	`selection = selection.ToUpper();`
"A local variable named ... is already defined in this scope"	Two cases declare the same name	Use a different name, or declare it once before the switch
An old answer shows up under a new choice	A case used `+=` on `result`, or didn't set it	Set `result` with `=` in every case
An option on the menu says it isn't an option	The menu line was added, but not the case	Add the case too
A second "Enter your selection" that does nothing	A case asks for a selection	Only the loop reads the selection

Check yourself

1. Why are `hasQuit` and `result` declared before the loop, but `selection` inside it?
2. Case Q has only `break;` in it. Does Q quit? Why or why not?
3. Which of these belong inside a case? `Console.Clear();` · `Console.Write("Bill amount: ");` · `result = $"You rolled a {roll}.";`
4. What two changes add option D to a menu?

Answers

1. `hasQuit` and `result` must survive from one pass to the next. `selection` is new every pass.
2. No. Inside a switch, `break` only leaves the switch. The loop needs `hasQuit = true;`.
3. `Console.Write("Bill amount: ");` and `result = ...;`. Clearing the screen belongs to the loop.
4. One line in the `MENU` text, and one new case.